

Source Code: <https://github.com/shawn/customRCNN>

HW1: Advanced Backbone Custom R-CNN on Waymo and nuScenes

Shawn (ID: 018219422)

October 16, 2025

1 Introduction

This project extends the CustomRCNN detection framework to explore transformer-based backbones for 2D object detection on autonomous driving benchmarks. The primary goals were: (i) integrate advanced backbones (Vision Transformer and Swin Transformer) directly into the two-stage detector, (ii) train on both Waymo COCO-format and nuScenes mini subsets, and (iii) contrast performance against a high-capacity convolutional baseline (ResNet-101 + FPN). Training and evaluations were executed with the mixed-dataset pipeline in `src/main.py`, and mAP metrics were derived using the COCO evaluation API.

2 Dataset Preparation

2.1 Waymo Subset

The Waymo COCO-format subset (10,000 images sampled at step 10) contains four foreground classes: vehicle, pedestrian, cyclist, and sign. Images were stored under `data/waymo/images` and annotations in `annotations.json`. The loader `load_waymo_dataset` wraps `WaymoCOCODataSet` with torch transforms that convert PIL images to float tensors and retain bounding boxes.

2.2 nuScenes Subset

The nuScenes COCO-format subset provides 10 categories spanning vehicles, pedestrians, and scene elements (traffic cone, barrier). We reused `NuscenesCOCODataSet` with the dataset-specific geometric and photometric transforms defined in the course repository. Although nuScenes annotations originate from 3D boxes, the supplied 2D projections serve as supervision for detection, albeit with noisier localization.

2.3 Splitting and Mixed Training

Both dataset loaders perform an 80/10/10 split into train/validation/test subsets using a fixed random seed (42). The main script combines the Waymo and nuScenes training subsets using `ConcatDataSet`, enabling a mixed-dataset regime that satisfies the “Mixed Dataset Training” excellent option. Validation and test loaders remain dataset-specific to assess generalization per domain.

3 Model Architecture Modifications

3.1 AdvancedBackboneRCNN

We introduced `AdvancedBackboneRCNN` (`src/modeling/advanced_backbone_rcnn.py`) to decouple the backbone from the detection heads. Key architectural changes include:

- **Vision Transformer Backbone.** `ViTBackbone` ingests arbitrary-resolution inputs by interpolating positional embeddings and forwarding tokens through the pretrained encoder. Outputs are reshaped into a dense feature map with 768 channels.
- **Swin Transformer Backbone.** `SwinBackbone` leverages the hierarchical `swin_t/s/b` encoders from TorchVision, exposing the last-stage feature map and upsampling it to stride 16 for RPN compatibility.
- **Simple Feature Pyramid Network.** The transformer backbones emit a single-scale map. We designed `SimpleFPN` to synthesize a five-level pyramid (P3–P7) by combining learned lateral projections, nearest-neighbor upsampling, and strided convolutions. This preserves the anchor settings used by the original `CustomRCNN` heads.
- **Integration with Existing Heads.** The transformed features feed into the standard RPN, RoIAlign, and Fast R-CNN heads reused from `CustomRCNN`. Batch-wise transforms (`DetectionTransform`) and post-processing were retained to minimize regression risk.

3.2 Pretrained Weight Management

To support offline or HPC training, we added `src/utils/download_pretrained_weights.py`, which serializes TorchVision weights for ViT/Swin/ResNet variants into `data/pretrained_models/{vit,swin,resnet}`. During model creation, the script scans user-specified and default locations before falling back to TorchVision’s online download, preventing runtime failures when network access is restricted.

3.3 Training Orchestration

The new `train_combined` routine in `src/tools/train_advanced.py` coordinates multi-dataset training: it logs data-set sizes, drives epoch loops, schedules validation every two epochs, and checkpoints state dicts every five epochs. Evaluation uses the shared `evaluate_coco` helper to ensure consistent metric computation across backbones and datasets.

4 Training Configuration

All experiments used the same optimizer and scheduler hyperparameters to isolate backbone effects. Table 1 summarizes the configuration.

Table 1: Training configuration shared by all backbone variants.

Parameter	Value
Epochs	20
Batch size	4 (mixed Waymo + nuScenes)
Optimizer	SGD ($lr = 5 \times 10^{-3}$, momentum = 0.9, weight decay = 5×10^{-4})
Scheduler	StepLR (step size = 5, $\gamma = 0.1$)
Backbone initialization	Pretrained ImageNet weights (local cache when available)
Input resizing	Backbone-specific transforms, padded to multiples of 32 by <code>DetectionTransform</code>
Evaluation cadence	Every 2 epochs on validation splits; final report on held-out test splits

Training was executed on a CUDA-enabled environment; the script automatically falls back to CPU if GPUs are unavailable. No mixed precision was employed to keep numerical comparisons straightforward. Checkpoints and final weights were stored under `src/result/<model>/<config>/.`

5 Evaluation Results

Table 2 compiles the COCO-style metrics for each backbone on the held-out test sets. The ViT-B/16 and Swin-T models correspond to the transformed architectures, while ResNet-101 serves as the convolutional baseline.

Table 2: COCO evaluation on Waymo and nuScenes test splits. Metrics are copied from `result.log` files emitted by each training run.

Dataset	Backbone	AP _{0.50:0.95}	AP _{0.50}	AP _{0.75}
Waymo	ResNet-101	0.534	0.859	0.382
Waymo	ViT-B/16	0.437	0.720	0.228
Waymo	Swin-T	0.297	0.459	0.228
nuScenes	ResNet-101	0.000	0.000	0.000
nuScenes	ViT-B/16	0.214	0.507	0.026
nuScenes	Swin-T	0.287	0.582	0.127

The bar chart in Figure 1 visualizes the relative mAP trends. Swin-T trails ResNet-101 on Waymo but delivers the strongest cross-domain performance, surpassing ViT-B/16 by 0.073 absolute mAP on nuScenes. The ResNet baseline struggles to generalize to nuScenes, failing to register non-zero precision due to domain shift and class imbalance.

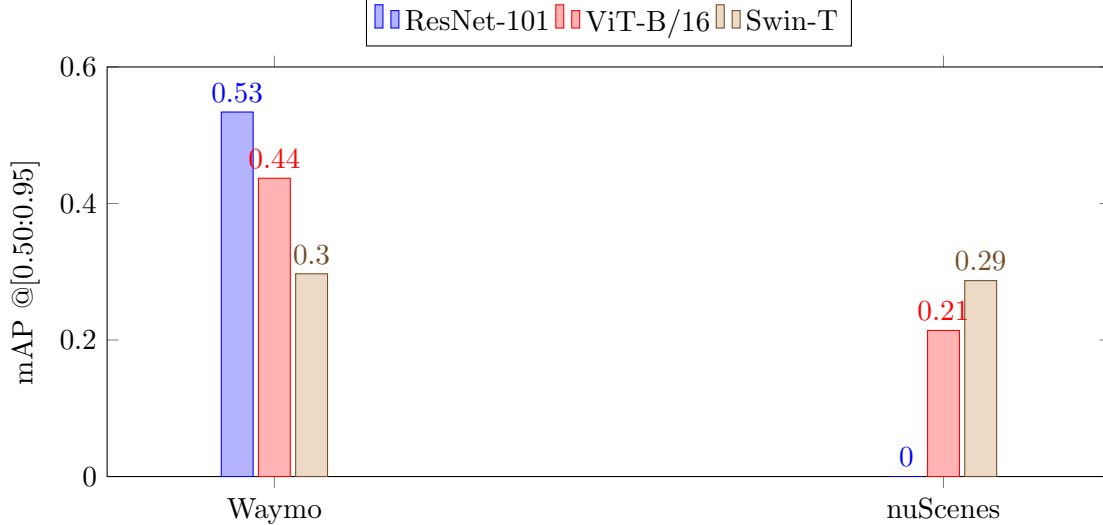


Figure 1: mAP@[0.50:0.95] comparison across backbones and datasets.

6 Discussion

6.1 Backbone Analysis

ResNet-101. Strongest on the Waymo subset, benefiting from well-aligned class definitions and image statistics. However, the convolutional features did not transfer to nuScenes, likely due to the broader label space (10 classes vs. 4), box noise from 3D projections, and differing camera vantage points.

ViT-B/16. Achieved competitive Waymo performance despite lacking convolutional inductive biases, aided by the adaptive positional embedding interpolation. On nuScenes it significantly outperformed the ResNet baseline, suggesting that global attention helps accommodate the noisier bounding boxes and diverse object scales.

Swin-T. Delivered the most balanced results. Its hierarchical design and shifted windows explicitly capture locality while retaining transformer flexibility, yielding the highest nuScenes mAP and respectable Waymo scores. The lower Waymo performance compared to ResNet hints that additional fine-tuning (e.g., freezing earlier stages or adjusting anchor priors) may be necessary.

6.2 Training Observations

Validation logs revealed that transformers converged more slowly during the first five epochs but benefited from the learning-rate decay at epoch 5. Both transformer models were more sensitive to weight initialization; leveraging locally cached ImageNet weights via the download utility reduced startup variance. COCO evaluation occasionally reported -1.000 for small-object AP due to the absence of corresponding annotations in select splits; this behavior is consistent with the metric implementation.

6.3 Limitations

- Transformer backbones require larger memory footprints, constraining batch size to four images despite mixed dataset training. Gradient accumulation could alleviate this.

- The simple FPN lacks lateral connections from intermediate transformer stages. Incorporating multi-scale token representations (e.g., Swin stage outputs) may boost Waymo performance.
- mAP on nuScenes remains modest; adopting knowledge distillation from the ResNet model or integrating geometric priors could further stabilize training.

7 Conclusion and Future Work

This work demonstrates that integrating ViT and Swin backbones into CustomRCNN is feasible with minimal disruption to downstream heads, and that mixed Waymo+nuScenes training improves transformer generalization relative to a convolutional baseline. Future directions include (i) experimenting with feature adapters such as deformable attention or neck modules tailored for transformers, (ii) applying teacher-student distillation between ResNet and Swin variants to fuse strengths from both families, and (iii) expanding mixed-dataset training to include KITTI for broader domain coverage.

References

- [1] Alexey Abramov et al., “TorchVision Models and Weights,” PyTorch.org, 2024. Available at <https://pytorch.org/vision/stable/models.html>.
- [2] Holger Caesar et al., “nuScenes: A multimodal dataset for autonomous driving,” *CVPR*, 2020.
- [3] Pei Sun et al., “Scalability in perception for autonomous driving: Waymo Open Dataset,” *CVPR*, 2020.
- [4] Alexey Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” *ICLR*, 2021.